## Buổi 5: Lưu trữ dữ liệu dài hạn

**Nội dung chính**

5.1. Lưu trữ dữ liệu dạng khóa - giá trị trên Android
5.2. Tạo cơ sở dữ liệu quan hệ cho ứng dụng trên Android
5.3. Quản lý dữ liệu với thư viện Room

**Mục tiêu cần đạt được**

Hiểu các phương pháp lưu trữ dữ liệu dài hạn trên Android, bao gồm sử dụng Shared Preferences, SQLite và Room.

Biết cách sử dụng và thao tác với Shared Preferences để lưu trữ dữ liệu dạng khóa - giá trị, thiết lập và thao tác cơ sở dữ liệu với SQLite.

Vận dụng các kỹ thuật lưu trữ và quản lý dữ liệu để xây dựng ứng dụng Android sử dụng cơ sở dữ liệu SQLite hoặc thư viện Room để tối ưu hóa việc lưu trữ và truy vấn dữ liệu.

### 5.1. Lưu trữ dữ liệu với Shared preferences

Việc lưu trữ dữ liệu sử dụng Shared preferences có thể được mô tả minh hoạ qua Hình 3. sau: giả sử người dùng đang chơi một trò chơi trên Android, trước khi chơi trò chơi người chơi đã lựa chọn các thông số của trò chơi như độ sáng trong trò chơi, mức độ âm lượng và độ khó. Sau khi chơi xong người chơi đã tắt trò chơi và có thể tiếp tục chơi vào ngày hôm sau. Shared Preferences cho phép lưu lại các các thông số người dùng đã thiết lập trước đó, khi người dùng chơi lại các thiết lập trước đó có thể tự động được cập nhật mà không cần phải thiết lập lại.

Shared Preferences bản chất là một file xml của ứng dụng. Nó lưu các dữ liệu thô dưới dạng các cặp khóa và giá trị (key-value) vào các tập tin của ứng dụng. Người lập trình cũng có thể chọn một chế độ lưu trữ riêng tư (PRIVATE) mà các ứng dụng khác không thể truy cập vào các tập tin này, do đó cách lưu trữ này là khá an toàn.

#### *5.1.1. Ghi dữ liệu vào Shared Preferences*

Để lưu ghi dữ liệu vào file lưu trữ của ứng dụng, người lập trình cần thực hiện lấy về đối tượng SharedPreferences bằng cách gọi tới phương thức getSharedPreferences(). Phương thức này gồm có 2 đối số là tên file XML và chế độ lưu trữ (thường sử dụng PRIVATE)

```
val shPre = this.getSharedPreferences("file_name",
Context.MODE_PRIVATE)
```

Tạo đối tượng Editor để sửa đổi dữ liệu như sau:

```
val editor = shPre.edit()
```

Sau khi khởi tạo được đối tượng Editor người lập trình có thể sử dụng các phương thức tương ứng với kiểu dữ liệu muốn ghi vào file như:

```
editor.putString("key", "value")  // ghi dữ liệu kiểu String
editor.putInt("key", 123)          // ghi dữ liệu kiểu Int
editor.putBoolean("key", true)     // ghi dữ liệu kiểu Boolean
editor.putFloat("key", 1.23f)      // ghi dữ liệu kiểu Float
editor.putLong("key", 123456789L)  // ghi dữ liệu kiểu Long
```

Tham số thứ nhất luôn là kiểu String. Ví dụ người lập trình thực hiện ghi dữ liệu như sau:

```
editor.putString("name", "Nguyen Van A")
editor.putBoolean("isNam", true)
```

Để thực hiện cập nhật dữ liệu vào file sau khi thực hiện các phương thức put thì phải gọi phương thức apply() hoặc commit() của đối tượng Editor để ghi dữ liệu vào file:

```
editor.apply()  // Không chặn UI, thay đổi được lưu bất đồng bộ
// hoặc
editor.commit() // Chặn UI, thay đổi được lưu đồng bộ
```

### *5.1.2. Đọc dữ liệu từ Shared Preferences*

Để đọc được dữ liệu từ SharedPreferences người lập trình tạo đối tượng SharedPreferences giống như ghi dữ liệu:

```
val shPre = this.getSharedPreferences("file_name",
Context.MODE_PRIVATE)
val editor = shPre.edit()
```

Sau đó sử dụng các phương thức đọc tương ứng với kiểu dữ liệu của đối tương Editor:

```
val valueInt = shPre.getInt("key", defaultValue)
// Lấy dữ liệu kiểu Int
val valueString = shPre.getString("key", "defaultValue")
// Lấy dữ liệu kiểu String
val valueBoolean = shPre.getBoolean("key", false)
// Lấy dữ liệu kiểu Boolean
```

### *5.1.3. Xoá dữ liệu SharedPreferences*

Phương remove() của đối tương Editor được sử dụng để xoá từng cặp dữ liệu trong SharedPreferences.

Ví dụ:

```
editor.remove("name")  // Xoá cặp có key là "name"
editor.remove("email") // Xoá cặp có key là "email"
editor.commit()        // Thực hiện xoá
```

Phương thức clear() của đối tượng Editor cho phép xoá toàn bộ dữ liệu trong SharedPreferences.

Ví dụ:

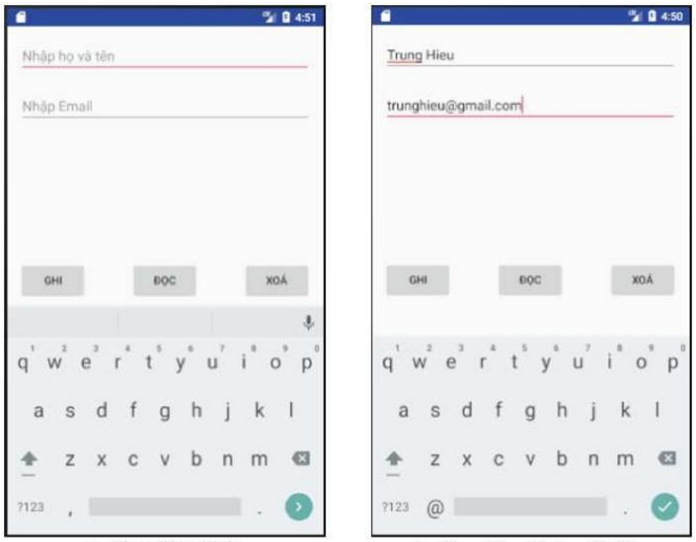
```
editor.clear()   // Xóa tất cả các cặp key-value trong
SharedPreferences
editor.commit()  // Thực hiện xoá
```

Thiết kế giao diện như hình (a). Xây dựng chương trình để khi người dùng nhập dữ liệu vào 2 đối tượng EditText rồi nhấn nút Ghi thì dữ liệu được ghi vào file Myprefs.xml trên hệ thống, khi nhấn nút Đọc thì đọc dữ liệu từ file Myprefs.xml hiển thị lên các

EditText tương ứng, khi nhấn nút Xoá thì xoá dữ liệu trên các EditText. Nếu người dùng tắt ứng dụng hoàn toàn và mở lại nó thì giá trị vừa nhập sẽ lưu lại trên các EditText.

Tạo project mới và soạn thảo nội dung các file như sau:

Bước 1. Soạn thảo file activiy_main.xml có nội dung như sau:




a. Giao diện chính b. Giao diện khi đọc dữ liệu

*Hình 5.1: Giao diện ứng dụng sử dụng Shared Preferences*

```
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin">

    <Button
        android:id="@+id/btnSave"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerVertical="true"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true"
        android:onClick="Save"
        android:text="Ghi" />

    <Button
        android:id="@+id/btnRetr"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerHorizontal="true"
        android:layout_centerVertical="true"
        android:onClick="Get"
        android:text="Đọc" />

    <Button
        android:id="@+id/btnClear"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignRight="@+id/etEmail"
        android:layout_centerVertical="true"
```

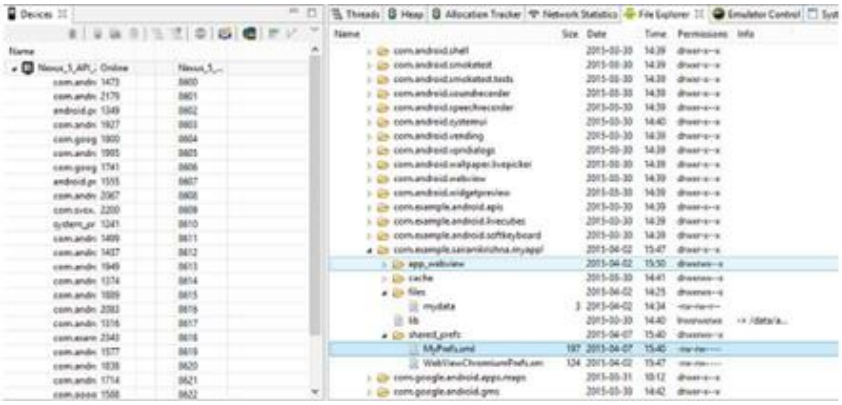
```
        android:layout_alignParentRight="true"
        android:layout_alignParentEnd="true"
        android:onClick="clear"
        android:text="Xoá" />

    <EditText
        android:id="@+id/etEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="Nhập Email"
        android:inputType="textEmailAddress"
        android:layout_below="@+id/etName"
        android:layout_marginTop="20dp"
        android:layout_alignParentRight="true"
        android:layout_alignParentEnd="true" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:ems="10"
        android:hint="Nhập họ và tên"
        android:inputType="text"
        android:layout_alignParentTop="true"
        android:layout_alignLeft="@+id/etEmail"
        android:layout_alignStart="@+id/etEmail" />
</RelativeLayout>
```

Bước 2: Soạn thảo file MainActivity.kt có nội dung như sau:

```
class MainActivity : Activity() {
    private lateinit var sharedPreferences: SharedPreferences
    private lateinit var name: TextView
    private lateinit var email: TextView

    companion object {
        const val myPreference = "mypref"
        const val Name = "nameKey"
        const val Email = "emailKey"
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        name = findViewById(R.id.etName)
        email = findViewById(R.id.etEmail)
        sharedPreferences = getSharedPreferences(myPreference,
        Context.MODE_PRIVATE)

        if (sharedPreferences.contains(Name)) {
            name.text = sharedPreferences.getString(Name, "")
        }
        if (sharedPreferences.contains(Email)) {
            email.text = sharedPreferences.getString(Email, "")
        }
    }

    // Phương thức ghi dữ liệu vào SharedPreferences
    fun save(view: View) {
        val n = name.text.toString()
```

```
        val e = email.text.toString()
        val editor = sharedPreferences.edit()
        editor.putString(Name, n)
        editor.putString(Email, e)
        editor.apply() // Có thể sử dụng apply() thay vì commit()
    }

    // Phương thức xóa dữ liệu trên các EditText
    fun clear(view: View) {
        name.text = ""
        email.text = ""
    }

    // Phương thức đọc dữ liệu từ SharedPreferences
    fun get(view: View) {
        if (sharedPreferences.contains(Name)) {
            name.text = sharedPreferences.getString(Name, "")
        }
        if (sharedPreferences.contains(Email)) {
            email.text = sharedPreferences.getString(Email, "")
        }
    }
}
```

Bước 3: Chạy ứng dụng và kiểm tra kết quả.



*Hình 5.2: Xem file Shared Preferences trên thiết bị*

## 5.2. Tạo cơ sở dữ liệu quan hệ cho ứng dụng trên Android

### *5.2.1. Giới thiệu về SQLite*

SQLite là một thư viện phần mềm cung cấp một cơ sở dữ liệu quan hệ (RDBMS) nhẹ và nhúng, cho phép lưu trữ và truy xuất dữ liệu dưới dạng tệp trong ứng dụng Android. SQLite được tích hợp sẵn trong Android, không yêu cầu cài đặt hoặc cấu hình riêng biệt, giúp các nhà phát triển dễ dàng quản lý dữ liệu mà không cần sử dụng một máy chủ bên ngoài.

Tại sao sử dụng SQLite trên Android?

- Nhẹ và hiệu quả: SQLite có kích thước nhỏ và tiêu thụ ít tài nguyên, rất phù hợp cho các thiết bị di động với bộ nhớ và năng lượng hạn chế.
- Dễ dàng tích hợp: SQLite là một phần của Android SDK, vì vậy bạn không cần phải thêm bất kỳ thư viện bên ngoài nào. Điều này làm giảm độ phức tạp trong việc phát triển ứng dụng.

- Hỗ trợ SQL chuẩn: SQLite hỗ trợ ngôn ngữ truy vấn SQL chuẩn, giúp các nhà phát triển dễ dàng viết và quản lý các truy vấn để thao tác với dữ liệu.
- Quản lý dữ liệu cục bộ: SQLite cho phép lưu trữ dữ liệu cục bộ trên thiết bị, giúp ứng dụng có thể hoạt động offline và cải thiện hiệu suất.
- Bảo mật: Dữ liệu trong SQLite có thể được mã hóa để bảo vệ thông tin nhạy cảm, điều này rất quan trọng đối với các ứng dụng yêu cầu bảo mật.

**Cách hoạt động của SQLite trong Android**

- Cơ sở dữ liệu nhúng: SQLite lưu trữ dữ liệu trong các tệp .db mà bạn có thể quản lý trực tiếp. Điều này có nghĩa là tất cả dữ liệu của ứng dụng đều nằm trong một tệp, giúp đơn giản hóa việc sao lưu và di chuyển dữ liệu.
- Lớp SQLiteOpenHelper: Để làm việc với SQLite, bạn thường sẽ tạo một lớp con của SQLiteOpenHelper. Lớp này giúp bạn quản lý vòng đời của cơ sở dữ liệu, bao gồm việc tạo và nâng cấp bảng khi cần thiết.
- Lớp SQLiteDatabase: Khi bạn đã khởi tạo cơ sở dữ liệu bằng SQLiteOpenHelper, bạn sẽ sử dụng lớp SQLiteDatabase để thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên cơ sở dữ liệu.
- Lớp Cursor: Khi bạn thực hiện các truy vấn dữ liệu, kết quả sẽ được trả về dưới dạng một đối tượng Cursor. Lớp này cho phép bạn duyệt qua kết quả và lấy dữ liệu cần thiết.

**Các lớp hỗ trợ làm việc với SQLite trong Android**

Android cung cấp một số lớp để tương tác với SQLite, bao gồm:

**SQLiteOpenHelper**: Đây là lớp cơ sở giúp quản lý cơ sở dữ liệu. Nó cung cấp các phương thức để tạo, nâng cấp và quản lý vòng đời của cơ sở dữ liệu.

Bạn cần kế thừa lớp này và triển khai các phương thức onCreate() và onUpgrade() để tạo và nâng cấp cơ sở dữ liệu.

```
class DatabaseHelper(context: Context) : SQLiteOpenHelper(context,
DATABASE_NAME, null, DATABASE_VERSION) {
    override fun onCreate(db: SQLiteDatabase) {
        // Tạo bảng
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int,
       newVersion: Int) {
        // Nâng cấp bảng
    }
}
```

**SQLiteDatabase**: Lớp này cung cấp các phương thức để thực hiện các thao tác với cơ sở dữ liệu, chẳng hạn như thêm, xóa, cập nhật và truy vấn dữ liệu.

Bạn có thể lấy đối tượng SQLiteDatabase từ lớp SQLiteOpenHelper để thực hiện các thao tác cần thiết.

```
val db: SQLiteDatabase = dbHelper.writableDatabase
```

Cursor:

**Lớp Cursor:** Đại diện cho kết quả của một truy vấn. Nó cho phép bạn duyệt qua các hàng dữ liệu mà truy vấn trả về. Bạn có thể sử dụng Cursor để lấy dữ liệu từ cơ sở dữ liệu một cách hiệu quả.

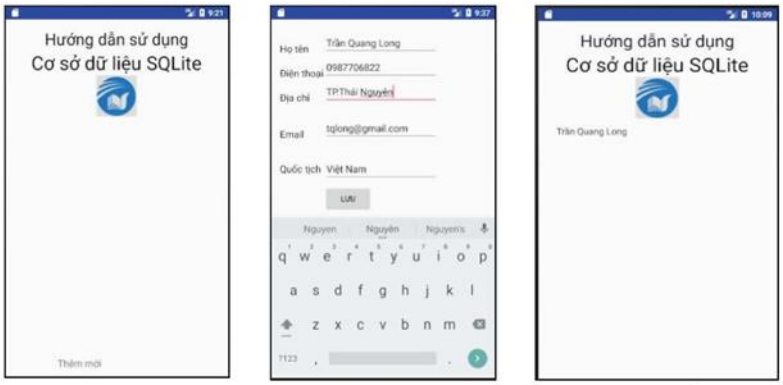
```
val cursor: Cursor = db.query(TABLE_NAME, null, null, null, null,
null, null)
```

**Cách sử dụng SQLite trong Android**

- Tạo lớp con của SQLiteOpenHelper: Như đã mô tả ở trên, kế thừa SQLiteOpenHelper và định nghĩa các phương thức cần thiết để quản lý cơ sở dữ liệu.
- Thực hiện các thao tác CRUD (Create, Read, Update, Delete): Sử dụng các phương thức của SQLiteDatabase để thêm, lấy, cập nhật và xóa dữ liệu.
- Quản lý vòng đời: Đảm bảo rằng bạn mở và đóng cơ sở dữ liệu đúng cách để tránh rò rỉ bộ nhớ.

### *5.2.2. Thao tác trên cơ sở dữ liệu quan hệ*

Ví dụ dưới đây sẽ minh hoạ các thao tác với CSDL SQLite. Thiết giao diện ứng dụng gồm giao diện chính như (a) gồm một menu option có nhãn "Thêm mới". Khi người dùng chọn menu thì hiển thị giao diện cho phép nhập dữ liệu về một contact như hình (b), giao diện này chứa 2 menu option là Edit_Contact và Delete_Contact, khi người dùng nhập các thông tin về một contact và nhấn nút Lưu thì lưu các thông tin về contact vào CSDL đồng thời trở lại màn hình chính và danh sách các contact đã nhập sẽ hiển thị trên màn hình chính như hình (c)




a. Giao diện chính ứng dụng

b. Giao diện khi click vào menu Thêm mới và nhập dữ liệu

c. Giao diện sau khi nhấn nút Lưu

*Hình 5.3: Ví dụ*

Tạo project mới và thực hiện các bước sau:

Bước 1: Tạo file layout activity_display_contact.xml định nghĩ giao diện nhập liệu cho contact như sau:

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/scrollView1"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    tools:context=".DisplayContact">

    <RelativeLayout
        android:layout_width="match_parent"
        android:layout_height="370dp"
        android:paddingBottom="@dimen/activity_vertical_margin"
        android:paddingLeft="@dimen/activity_horizontal_margin"
        android:paddingRight="@dimen/activity_horizontal_margin"
        android:paddingTop="@dimen/activity_vertical_margin">

        <EditText
            android:id="@+id/editTextName"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignParentLeft="true"
            android:layout_marginTop="5dp"
            android:layout_marginLeft="82dp"
            android:ems="10"
            android:inputType="text" />

        <EditText
            android:id="@+id/editTextEmail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignLeft="@+id/editTextStreet"
            android:layout_below="@+id/editTextStreet"
            android:layout_marginTop="22dp"
            android:ems="10"
            android:inputType="textEmailAddress" />

        <TextView
            android:id="@+id/textView1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignBottom="@+id/editTextName"
            android:layout_alignParentLeft="true"
            android:text="Họ tên"
            android:textAppearance="?android:attr/textAppearanceMedium"
        />

        <Button
            android:id="@+id/button1"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignLeft="@+id/editTextCity"
            android:layout_alignParentBottom="true"
            android:layout_marginBottom="28dp"
            android:onClick="run"
            android:text="Luu" />

        <TextView
            android:id="@+id/textView2"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignBottom="@+id/editTextEmail"
            android:layout_alignLeft="@+id/textView1"
```

```
            android:text="Email"

android:textAppearance="?android:attr/textAppearanceMedium" />

        <TextView
            android:id="@+id/textView5"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignBottom="@+id/editTextPhone"
            android:layout_alignLeft="@+id/textView1"
            android:text="Điện thoại"

android:textAppearance="?android:attr/textAppearanceMedium" />

        <TextView
            android:id="@+id/textView4"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_above="@+id/editTextEmail"
            android:layout_alignLeft="@+id/textView5"
            android:text="Địa chỉ"

android:textAppearance="?android:attr/textAppearanceMedium" />

        <EditText
            android:id="@+id/editTextCity"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignRight="@+id/editTextName"
            android:layout_below="@+id/editTextEmail"
            android:layout_marginTop="30dp"
            android:ems="10"
            android:inputType="text" />

        <TextView
            android:id="@+id/textView3"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignBaseline="@+id/editTextCity"
            android:layout_alignBottom="@+id/editTextCity"
            android:layout_alignParentLeft="true"
            android:layout_toLeftOf="@+id/editTextEmail"
            android:text="Quốc tịch"

android:textAppearance="?android:attr/textAppearanceMedium" />

        <EditText
            android:id="@+id/editTextStreet"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_alignLeft="@+id/editTextName"
            android:layout_below="@+id/editTextPhone"
            android:ems="10"
            android:inputType="text">
            <requestFocus />
        </EditText>

        <EditText
            android:id="@+id/editTextPhone"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
            android:layout_alignLeft="@+id/editTextStreet"
            android:layout_below="@+id/editTextName"
            android:ems="10"
            android:inputType="phone|text" />
    </RelativeLayout>
</ScrollView>
```

Bước 2: File layout activity_main.xml chứa giao diện chính có nội dung như sau:

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    android:paddingBottom="@dimen/activity_vertical_margin"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_centerHorizontal="true"
        android:textSize="30sp"
        android:text="Hướng dẫn sử dụng" />

    <TextView
        android:id="@+id/textView2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView"
        android:layout_centerHorizontal="true"
        android:textSize="35sp"
        android:textColor="#ff16ff01"
        android:text="Cơ sở dữ liệu SQLite" />

    <ImageView
        android:id="@+id/imageView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/textView2"
        android:layout_centerHorizontal="true"
        android:src="@drawable/logo" />

    <ScrollView
        android:id="@+id/scrollView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+id/imageView"
        android:layout_alignParentLeft="true"
        android:layout_alignParentStart="true"
        android:layout_alignParentBottom="true"
        android:layout_alignParentRight="true"
        android:layout_alignParentEnd="true">

        <ListView
            android:id="@+id/listView1"
            android:layout_width="match_parent"
```

```
            android:layout_height="wrap_content"
            android:layout_centerHorizontal="true"
            android:layout_centerVertical="true" />
    </ScrollView>
</RelativeLayout>
```

Bước 3: Tạo file res/menu/main_menu.xml định nghĩa menu option cho giao diện chính của ứng dụng:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/item1"
        android:icon="@drawable/add"
        android:title="Thêm mới" />
</menu>
```

Bước 4: Tạo file res/menu/display_contact.xml định nghĩa các menu option cho giao diện hiển thị và nhập dữ liệu cho một contact:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/Edit_Contact"
        android:orderInCategory="100"
        android:title="Sửa" />

    <item
        android:id="@+id/Delete_Contact"
        android:orderInCategory="100"
        android:title="Xoá" />
</menu>
```

Bước 5: File AndroidManifest.xml khai báo Activity DisplayCon- tact:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.sairamkrishna.myapplication">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">

        <activity
            android:name=".MainActivity"
            android:label="@string/app_name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity android:name=".DisplayContact" />
    </application>
</manifest>
```

Bước 6: Tạo lớp truy cập và quản lý CSDL DBHelper.kt có nội dung như sau:

```
import android.content.ContentValues
import android.content.Context
import android.database.Cursor
import android.database.DatabaseUtils
```

```
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class    DBHelper(context:    Context)    :    SQLiteOpenHelper(context,
DATABASE_NAME, null, 1) {

    companion object {
        const val DATABASE_NAME = "MyDBName.db"
        const val CONTACTS_TABLE_NAME = "contacts"
        const val CONTACTS_COLUMN_ID = "id"
        const val CONTACTS_COLUMN_NAME = "name"
        const val CONTACTS_COLUMN_EMAIL = "email"
        const val CONTACTS_COLUMN_STREET = "street"
        const val CONTACTS_COLUMN_CITY = "place"
        const val CONTACTS_COLUMN_PHONE = "phone"
    }

    override fun onCreate(db: SQLiteDatabase) {
        val createTable = ("CREATE TABLE $CONTACTS_TABLE_NAME (" +
                "$CONTACTS_COLUMN_ID INTEGER PRIMARY KEY, " +
                "$CONTACTS_COLUMN_NAME TEXT, " +
                "$CONTACTS_COLUMN_PHONE TEXT, " +
                "$CONTACTS_COLUMN_EMAIL TEXT, " +
                "$CONTACTS_COLUMN_STREET TEXT, " +
                "$CONTACTS_COLUMN_CITY TEXT)")
        db.execSQL(createTable)
    }

    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int,
newVersion:
       Int) {
        db.execSQL("DROP TABLE IF EXISTS $CONTACTS_TABLE_NAME")
        onCreate(db)
    }

    // Phương thức chèn một bản ghi vào CSDL
    fun insertContact(name: String, phone: String, email: String,
      street: String, place: String): Boolean {
        val db = this.writableDatabase
        val contentValues = ContentValues().apply {
            put(CONTACTS_COLUMN_NAME, name)
            put(CONTACTS_COLUMN_PHONE, phone)
            put(CONTACTS_COLUMN_EMAIL, email)
            put(CONTACTS_COLUMN_STREET, street)
            put(CONTACTS_COLUMN_CITY, place)
        }
        db.insert(CONTACTS_TABLE_NAME, null, contentValues)
        db.close()
        return true
    }

    // Chọn một bản ghi từ bảng contacts với id chỉ định
    fun getData(id: Int): Cursor {
        val db = this.readableDatabase
        return db.rawQuery("SELECT * FROM $CONTACTS_TABLE_NAME WHERE
       $CONTACTS_COLUMN_ID = ?", arrayOf(id.toString()))
    }

    // Lấy số lượng dòng của bảng contacts
    fun numberOfRows(): Int {
        val db = this.readableDatabase
```

```
        return DatabaseUtils.queryNumEntries(db,
       CONTACTS_TABLE_NAME).toInt()
    }

    // Phương thức cập nhật dữ liệu cho một bản ghi trong bảng contacts
    fun updateContact(id: Int, name: String, phone: String, email:
String,
       street: String, place: String): Boolean {
        val db = this.writableDatabase
        val contentValues = ContentValues().apply {
            put(CONTACTS_COLUMN_NAME, name)
            put(CONTACTS_COLUMN_PHONE, phone)
            put(CONTACTS_COLUMN_EMAIL, email)
            put(CONTACTS_COLUMN_STREET, street)
            put(CONTACTS_COLUMN_CITY, place)
        }
         db.update(CONTACTS_TABLE_NAME, contentValues,
      "$CONTACTS_COLUMN_ID
        = ?", arrayOf(id.toString()))
        db.close()
        return true
    }

    // Phương thức xoá một bản ghi trong bảng contacts
    fun deleteContact(id: Int): Int {
        val db = this.writableDatabase
        return db.delete(CONTACTS_TABLE_NAME, "$CONTACTS_COLUMN_ID = ?",
        arrayOf(id.toString()))
    }

    // Phương thức lấy thông tin về tất cả các contact
    fun getAllContacts(): ArrayList<String> {
        val arrayList = ArrayList<String>()
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM $CONTACTS_TABLE_NAME", null)
        if (cursor.moveToFirst()) {
            do {
                arrayList.add(cursor.
               getString(cursor.getColumnIndex(CONTACTS_COLUMN_NAME)))
            } while (cursor.moveToNext())
        }
        cursor.close()
        db.close()
        return arrayList
    }
}
```

Bước 7: Tạo lớp DisplayContact.kt xử lý dữ liệu trên giao diện nhập liệu cho các contact:

```
class DisplayContact : Activity() {
    var fromWhereIAmComing = 0
    private lateinit var mydb: DBHelper
    private lateinit var name: TextView
    private lateinit var phone: TextView
    private lateinit var email: TextView
    private lateinit var street: TextView
    private lateinit var place: TextView
    var idToUpdate = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
setContentView(R.layout.activity_display_contact)

name = findViewById(R.id.editTextName)
phone = findViewById(R.id.editTextPhone)
email = findViewById(R.id.editTextStreet)
street = findViewById(R.id.editTextEmail)
place = findViewById(R.id.editTextCity)

// Tạo đối tượng truy cập CSDL
mydb = DBHelper(this)

// Đọc giá trị biến id contact được chọn từ Activity
//MainActivity gửi sang
val extras = intent.extras
if (extras != null) {
    val value = extras.getInt("id")
    if (value > 0) {
        // Chọn bản ghi có id vừa nhận được
        val rs = mydb.getData(value)
        idToUpdate = value // đánh dấu id được chọn
        rs.moveToFirst()

        // Lấy thông tin của bản ghi có id được chọn
        val nam = rs.getString(
           rs.getColumnIndex
           (DBHelper.CONTACTS_COLUMN_NAME))
        val phon = rs.getString(
           rs.getColumnIndex
           DBHelper.CONTACTS_COLUMN_PHONE))
        val emai = rs.getString(
           rs.getColumnIndex
           (DBHelper.CONTACTS_COLUMN_EMAIL))
        val stree = rs.getString(
           rs.getColumnIndex
           (DBHelper.CONTACTS_COLUMN_STREET))
        val plac = rs.getString(
            rs.getColumnIndex
           (DBHelper.CONTACTS_COLUMN_CITY))
        // Đóng con trỏ
        if (!rs.isClosed) {
            rs.close()
        }
        // Ẩn button có nhãn Lưu
        val b = findViewById<Button>(R.id.button1)
        b.visibility = View.INVISIBLE
        name.text = nam
        name.isFocusable = false
        name.isClickable = false

        phone.text = phon
        phone.isFocusable = false
        phone.isClickable = false

        email.text = emai
        email.isFocusable = false
        email.isClickable = false

        street.text = stree
        street.isFocusable = false
        street.isClickable = false
```

```
            place.text = plac
            place.isFocusable = false
            place.isClickable = false
        }
    }
}

// Tạo menu option cho phép chỉnh sửa và xoá một contact được chọn
override fun onCreateOptionsMenu(menu: Menu): Boolean {
    val extras = intent.extras
    if (extras != null) {
        val value = extras.getInt("id")
        if (value > 0) {
            menuInflater.inflate(R.menu.display_contact, menu)
        } else {
            menuInflater.inflate(R.menu.menu_main, menu)
        }
    }
    return true
}
// Xử lý sự kiện khi các menu option được chọn
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.Edit_Contact -> {
            val b = findViewById<Button>(R.id.button1)
            b.visibility = View.VISIBLE

            name.isEnabled = true
            name.isFocusableInTouchMode = true
            name.isClickable = true

            phone.isEnabled = true
            phone.isFocusableInTouchMode = true
            phone.isClickable = true

            email.isEnabled = true
            email.isFocusableInTouchMode = true
            email.isClickable = true

            street.isEnabled = true
            street.isFocusableInTouchMode = true
            street.isClickable = true

            place.isEnabled = true
            place.isFocusableInTouchMode = true
            place.isClickable = true
            true
        }
        R.id.Delete_Contact -> {
            // Nếu chọn menu Xoá Contact thì hiển thị hộp thoại
            //xác nhận
            AlertDialog.Builder(this)
                .setMessage("Bạn có chắc chắn muốn xoá contact?")
                .setPositiveButton("Yes") { _, _ ->
                    mydb.deleteContact(idToUpdate)
                    Toast.makeText(this, "Xoá thành công",
                    Toast.LENGTH_SHORT).show()
                    val intent = Intent(this,
                     MainActivity::class.java)
                    startActivity(intent)
                }
```

```
                    .setNegativeButton("No", null)
                    .create()
                    .show()
                true
            }
            else -> super.onOptionsItemSelected(item)
        }
    }

    // Phương thức xử lý sự kiện khi Activity MainActivity
    fun run(view: View) {
        val extras = intent.extras
        if (extras != null) {
            val value = extras.getInt("id")
            if (value > 0) {
                // Cập nhật contact
                if (mydb.updateContact(
                        idToUpdate,
                        name.text.toString(),
                        phone.text.toString(),
                        email.text.toString(),
                        street.text.toString(),
                        place.text.toString()
                    )
                ) {
                    Toast.makeText(this, "Cập nhật thành công",
                    Toast.LENGTH_SHORT).show()
                     val intent = Intent(this,
                         MainActivity::class.java)
                    startActivity(intent)
                } else {
                    Toast.makeText(this, "Cập nhật không thành công",
                    Toast.LENGTH_SHORT).show()
                }
            } else {
                // Thêm mới contact
                if (mydb.insertContact(
                        name.text.toString(),
                        phone.text.toString(),
                        email.text.toString(),
                        street.text.toString(),
                        place.text.toString()
                    )
                ) {
                    Toast.makeText(this, "Đã chèn một bản ghi mới",
                    Toast.LENGTH_SHORT).show()
                } else {
                    Toast.makeText(this, "Chèn bản ghi không thành
                   công",
                    Toast.LENGTH_SHORT).show()
                }
                val intent = Intent(this, MainActivity::class.java)
                startActivity(intent)
            }
        }
    }
}
```

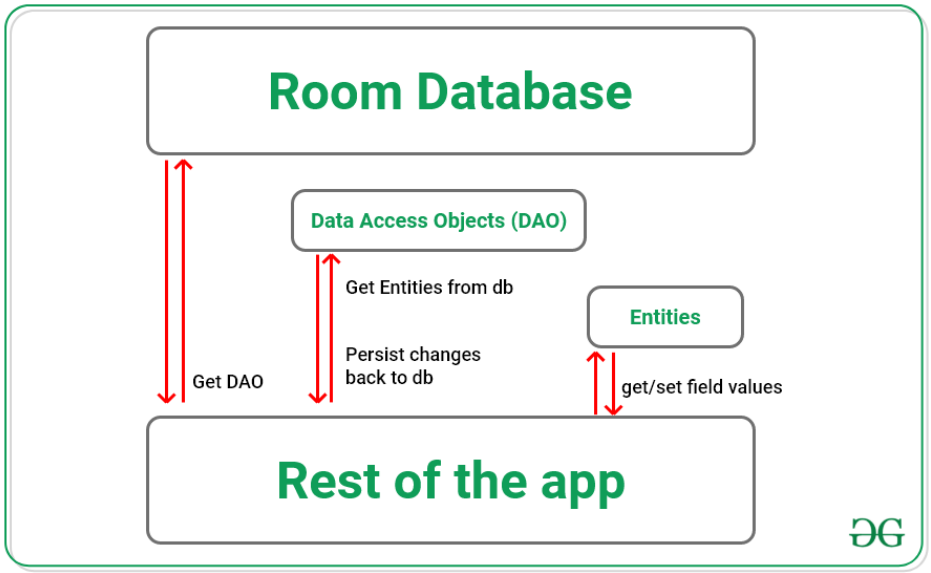
Bước 8: Soạn thảo file chương trình MainActivity.kt chính như sau:

```
class MainActivity : ActionBarActivity() {
    companion object {
```

```
        const val EXTRA_MESSAGE = "MESSAGE"
    }
    private lateinit var listView: ListView
    private lateinit var myDb: DBHelper

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        myDb = DBHelper(this)
        val arrayList = myDb.getAllCotacts()
        val arrayAdapter = ArrayAdapter(this,
        android.R.layout.simple_list_item_1, arrayList)
        listView = findViewById(R.id.listView1)
        listView.adapter = arrayAdapter
        listView.setOnItemClickListener {
            parent, view, position, id ->
            val idToSearch = position + 1
            val dataBundle = Bundle().apply {
                putInt("id", idToSearch)
            }
            val intent = Intent(applicationContext,
             DisplayContact::class.java).apply {
                putExtras(dataBundle)
            }
            startActivity(intent)
        }
    }

    override fun onCreateOptionsMenu(menu: Menu): Boolean {
        menuInflater.inflate(R.menu.menu_main, menu)
        return true
    }

    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        super.onOptionsItemSelected(item)
        return when (item.itemId) {
            R.id.item1 -> {
                val dataBundle = Bundle().apply {
                    putInt("id", 0)
                }
                val intent = Intent(applicationContext,
                    DisplayContact::class.java).apply {
                    putExtras(dataBundle)
                }
                startActivity(intent)
                true
            }
            else -> super.onOptionsItemSelected(item)
        }
    }
}
```

## 5.3. Quản lý dữ liệu với thư viện Room

### *5.3.1. Giới thiệu về Room*




*Hình 5.4: Room*

Room là một thư viện trong Android được phát triển bởi Google để giúp quản lý cơ sở dữ liệu SQLite một cách hiệu quả và dễ dàng hơn. Thư viện này sử dụng các lớp Java hoặc Kotlin để tương tác với cơ sở dữ liệu, thay vì phải viết các câu lệnh SQL thủ công. Room cung cấp một API đơn giản và trực quan, giúp lập trình viên tránh được nhiều lỗi khi làm việc với SQLite, đồng thời cải thiện khả năng bảo trì mã nguồn.

Các lợi ích của việc sử dụng Room:

- **Tránh lỗi SQL thủ công**: Room tự động sinh mã SQL và thực hiện việc tối ưu hóa các truy vấn.
- **Quản lý bộ nhớ hiệu quả**: Room sử dụng một cơ chế tái sử dụng kết quả truy vấn và ánh xạ đối tượng, giảm thiểu bộ nhớ.
- **Hỗ trợ LiveData và Coroutine**: Room tích hợp tốt với các thành phần khác của Android như LiveData và Coroutine, giúp dễ dàng cập nhật UI khi dữ liệu thay đổi.

### *5.3.2. Cài đặt và cấu hình Room trong dự án Android*

Để sử dụng Room trong dự án Android, bạn cần làm theo các bước sau:

**Thêm dependencies vào** build.gradle

Mở file build.gradle (Module: app) và thêm các dòng sau vào phần dependencies:

```
dependencies {
    def room_version = "2.5.0"  // Phiên bản Room
    implementation "androidx.room:room-runtime:$room_version"
    annotationProcessor "androidx.room:room-compiler:$room_version"
// Đối với Java
```

```
    kapt "androidx.room:room-compiler:$room_version"  // Đối với
Kotlin

    // Thêm dependency cho LiveData và Coroutine (nếu cần)
    implementation "androidx.room:room-ktx:$room_version"  // Dành cho
Kotlin
}
```

**Tạo lớp RoomDatabase**

Lớp RoomDatabase là lớp trừu tượng dùng để quản lý cơ sở dữ liệu và kết nối với các DAO (Data Access Object). Mỗi ứng dụng chỉ có một lớp RoomDatabase.

Ví dụ về lớp RoomDatabase:

```
@Database(entities = [User::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

Trong đó:

- entities: Danh sách các Entity (tương đương các bảng trong cơ sở dữ liệu).
- version: Phiên bản của cơ sở dữ liệu. Khi thay đổi cấu trúc cơ sở dữ liệu, bạn cần tăng số phiên bản này.

**Tạo Entity**

Lớp Entity đại diện cho bảng trong cơ sở dữ liệu. Các trường trong lớp sẽ được ánh xạ với các cột trong bảng.

Ví dụ về Entity:

```
@Entity(tableName = "user_table")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int,
    val name: String,
    val age: Int
)
```

- @Entity: Đánh dấu lớp là một Entity (tương ứng với một bảng trong cơ sở dữ liệu).
- @PrimaryKey: Đánh dấu cột là khóa chính (Primary Key).
- autoGenerate = true: Chế độ tự động sinh giá trị cho khóa chính.

### *5.3.3. Các thành phần chính của Room: Entity, DAO, Database*

**Entity:**

Mỗi lớp Entity đại diện cho một bảng trong cơ sở dữ liệu. Các thuộc tính trong lớp sẽ ánh xạ với các cột trong bảng. Các annotation phổ biến:

- @Entity: Đánh dấu lớp là một bảng.
- @PrimaryKey: Đánh dấu khóa chính.
- @ColumnInfo: Xác định tên cột trong bảng (nếu khác với tên thuộc tính).

Ví dụ về một Entity trong Room:

```
@Entity(tableName = "user_table")
data class User(
    @PrimaryKey(autoGenerate = true) val id: Int,
    val name: String,
    val age: Int
)
```

**DAO (Data Access Object):**

DAO là nơi bạn định nghĩa các phương thức để tương tác với cơ sở dữ liệu. Bạn có thể thực hiện các thao tác như chèn (insert), cập nhật (update), xóa (delete) và truy vấn (query) dữ liệu.

Một ví dụ về DAO:

```
@Dao
interface UserDao {
    @Insert
    suspend fun insert(user: User)

    @Update
    suspend fun update(user: User)

    @Delete
    suspend fun delete(user: User)

    @Query("SELECT * FROM user_table")
    suspend fun getAllUsers(): List<User>
}
```

**RoomDatabase:**

Lớp này quản lý kết nối giữa `DAO` và cơ sở dữ liệu SQLite. Nó tạo ra các đối tượng `DAO` để bạn có thể thực hiện các thao tác với dữ liệu.

```
val db = Room.databaseBuilder(context, AppDatabase::class.java,
"app_database")
     .build()
```

- databaseBuilder: Xây dựng cơ sở dữ liệu Room.
- AppDatabase::class.java: Lớp RoomDatabase bạn đã định nghĩa.
- "app_database": Tên cơ sở dữ liệu.

### *5.3.4. Thực hiện các thao tác cơ bản với Room*

**Thêm dữ liệu (Insert):**

Để thêm dữ liệu vào cơ sở dữ liệu, bạn sử dụng phương thức insert trong DAO.

```
val user = User(name = "John Doe", age = 30)
userDao.insert(user)
```

Phương thức insert sẽ thêm một bản ghi mới vào bảng user_table.

**Cập nhật dữ liệu (Update):**

Phương thức `update` trong DAO sẽ cập nhật một bản ghi có sẵn trong cơ sở dữ liệu.

```
val updatedUser = User(id = 1, name = "John Smith", age = 31)
```

```
userDao.update(updatedUser)
```
update sẽ tìm và thay thế bản ghi có id = 1 bằng bản ghi mới.

**Xóa dữ liệu (Delete):**

Phương thức delete trong DAO sẽ xóa một bản ghi.

```
val userToDelete = User(id = 1, name = "John Smith", age = 31)
userDao.delete(userToDelete)
```
delete sẽ xóa bản ghi có id = 1.

**Truy vấn dữ liệu (Query):**

Để truy vấn dữ liệu, bạn sử dụng phương thức `@Query` trong DAO. Ví dụ, để lấy tất cả người dùng:

```
val users = userDao.getAllUsers()
```
@Query("SELECT * FROM user_table") thực hiện truy vấn SQL để lấy tất cả người dùng từ bảng user_table.

Để sử dụng kết quả truy vấn, bạn có thể xử lý dữ liệu trong các Coroutine hoặc LiveData.

```
val users = liveData {
    emit(userDao.getAllUsers())
}
```

## Bài tập bổ trợ

**Bài tập 1: Lưu trữ và đọc dữ liệu với SharedPreferences**

Tạo một ứng dụng lưu trữ tên người dùng vào SharedPreferences khi nhấn nút "Lưu".

Hiển thị tên người dùng được lưu khi ứng dụng được mở lại.

**Bài tập 2: Lưu trữ dữ liệu sinh viên bằng SQLite**

Tạo bảng SQLite lưu thông tin sinh viên gồm id, name, age.

Thêm chức năng:

- Thêm một sinh viên mới.
- Hiển thị danh sách sinh viên trên giao diện.
- Xóa một sinh viên theo id.

**Bài tập 3: Sử dụng Room để quản lý dữ liệu**

Sử dụng thư viện Room để tạo bảng sinh viên (gồm id, name, age).

Thực hiện các thao tác CRUD:

- Thêm, xóa và sửa thông tin sinh viên.
- Hiển thị danh sách sinh viên từ cơ sở dữ liệu trên RecyclerView.

**Bài tập 4: Lưu trữ dữ liệu danh sách dạng JSON**

Lưu danh sách sản phẩm (tên và giá) vào bộ nhớ trong dưới dạng file JSON.

Đọc dữ liệu từ file JSON và hiển thị trên RecyclerView.

**Bài tập 5: Tạo ứng dụng ghi chú sử dụng Room**

Tạo một ứng dụng quản lý ghi chú với các chức năng:

- Thêm ghi chú mới với tiêu đề và nội dung.
- Hiển thị danh sách các ghi chú.
- Xóa một ghi chú khi nhấn giữ vào nó.

Lưu trữ dữ liệu ghi chú bằng Room.

**Bài tập mẫu**

**Bài tập 1: Lưu và đọc dữ liệu với SharedPreferences**

Mục tiêu: Lưu và hiển thị tên người dùng.

**Lưu dữ liệu:**

```
val sharedPreferences = getSharedPreferences("MyPrefs",
Context.MODE_PRIVATE)
val editor = sharedPreferences.edit()
editor.putString("username", "Nguyễn Văn A")
editor.apply()
```

**Đọc dữ liệu:**

```
val sharedPreferences = getSharedPreferences("MyPrefs",
Context.MODE_PRIVATE)
val username = sharedPreferences.getString("username", "Tên mặc định")
textView.text = "Xin chào, $username!"
```

**Bài tập 2: CRUD với Room**

Yêu cầu: Thực hiện CRUD với bảng sinh viên.

**Entity:**

```
@Entity
data class Student(
    @PrimaryKey(autoGenerate = true) val id: Int,
    val name: String,
    val age: Int
)
```

**DAO:**

```
@Dao
interface StudentDao {
    @Insert
    fun insert(student: Student)

    @Delete
    fun delete(student: Student)

    @Query("SELECT * FROM Student")
    fun getAllStudents(): List<Student>
}
```

**Database:**

```
@Database(entities = [Student::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun studentDao(): StudentDao
}
```

**Sử dụng trong ViewModel:**

```
class StudentViewModel(application: Application) :
AndroidViewModel(application) {
    private val db = Room.databaseBuilder(
        application,
        AppDatabase::class.java,
        "student-database"
    ).build()

    val students = MutableLiveData<List<Student>>()

    fun loadStudents() {
        students.value = db.studentDao().getAllStudents()
    }

    fun addStudent(name: String, age: Int) {
        db.studentDao().insert(Student(0, name, age))
        loadStudents()
    }
}
```